



SocialKids

Isla Conecta

Manual técnico — SocialKids

Manual técnico

Aplicación Android educativa para niñas y niños de 8 a 12 años
Empatía · Comunicación asertiva · Habilidades sociales

Version 1.0.0 · Kotlin · Jetpack Compose · Room · Sin conexión



1. Ficha técnica

Elemento	Valor
Lenguaje	Kotlin 2.0.21
Interfaz	Jetpack Compose (BOM 2024.12.01), Material 3
Navegación	Navigation Compose 2.8.5
Persistencia	Room 2.6.1 con KSP
Preferencias	DataStore Preferences 1.1.1
Asincronía	Coroutines 1.9.0, Flow / StateFlow
Arquitectura	MVVM + Repository, capas data / domain / ui
Build	Gradle 8.11.1 (Kotlin DSL) + AGP 8.7.3
JDK	17
minSdk / targetSdk / compileSdk	24 / 35 / 35
Permisos	ninguno
Idioma	español(resourceConfigurations = ["es"])

Todas las versiones están fijadas en gradle/libs.versions.toml. No se usan versiones dinámicas.

2. Estructura de paquetes

com.socialkids.app	
■■■■ SocialKidsApp.kt	Application + Contenedor (inyección manual)
■■■■ MainActivity.kt	Única Activity, splash + tema + navegación
■■■■ data	
■ ■■■■ local	
■ ■ ■■■■ SocialKidsDatabase.kt	Room, 7 entidades, versión 1
■ ■ ■■■■ entity/Entidades.kt	perfil, progreso, intento, carta, insignia, animo, visita
■ ■ ■■■■ dao/Daos.kt	6 DAO como interfaces
■ ■■■■ seed	Contenido del juego (24 misiones, 25 cartas, retos)
■ ■■■■ repository	
■ ■■■■ JuegoRepository.kt	Única puerta a los datos del juego
■ ■■■■ AjustesRepository.kt	DataStore de accesibilidad y confort
■■■■ domain	
■ ■■■■ model	Zona, Misión, Carta, Insignia, Resultado, EstadoJuego
■ ■■■■ engine	6 motores puros y testeables
■ ■■■■ usecase	Progreso, Desbloqueo, Insignias, Estadísticas, Racha
■■■■ ui	
■ ■■■■ art	Ilustraciones Canvas: Nima, Rostro, Avatares, Figuras, Escenarios
■ ■■■■ components	Botones, tarjetas, barras, arrastre
■ ■■■■ screens	20 archivos de pantalla y actividad
■ ■■■■ theme	Paleta, tipografía, esquemas claro/oscurο
■ ■■■■ navigation	Rutas y NavHost
■ ■■■■ JuegoViewModel.kt	Estado global del juego
■ ■■■■ ActividadViewModel.kt	Estado de la misión en curso
■■■■ util	Reloj (abstraído para tests) y Retroalimentación

Regla de dependencias: ui → domain ← data. El paquete domain **no importa nada de Android**, lo que permite ejecutar sus pruebas en la JVM sin emulador.

3. Los seis motores

Todos viven en domain/engine, son object sin estado y no conocen Compose ni Android.

3.1 RostroEngine

```
data class RostroConfig(cejas: Int, ojos: Int, boca: Int, energia: Int, extras: Set<RasgoExtra>)
```

Cada eje va de 0 a 100. `puntajeEje` convierte la distancia al objetivo en puntuación, con una tolerancia de 60 unidades: a partir de ahí el eje puntúa 0.

```
puntaje = (boca·0.32 + cejas·0.28 + ojos·0.22 + energia·0.18) · 0.85
          + puntajeExtras · 0.15
```

`puntajeExtras` premia los rasgos acertados y penaliza los sobrantes a media unidad cada uno. El desglose por eje se devuelve en `ResultadoActividad.detalles`, y `ejeMasLejano` genera la pista («prueba a mover la boca: ahí es donde tu rostro se aleja más»).

3.2 EscuchaEngine

`puntajeMemoria` es un **F1** clásico entre lo marcado y lo realmente dicho:

```
precision = aciertos / seleccionados
cobertura = aciertos / reales
F1         = 2·precision·cobertura / (precision + cobertura)
```

El resultado final combina memoria (60 %) con la calidad del `TipoRespuesta` elegido (40 %). Los seis tipos tienen calidad fija: reflejar 100, preguntar 88, validar 82, aconsejar rápido 45, desviar 25, juzgar 10.

3.3 PuenteEngine

Un mapa `Tablon → PiezaPuente?`. Cada tablón acertado aporta un tercio, y cada pieza trampa colocada resta 6 puntos. `estabilidad()` devuelve 0f–1f y alimenta directamente el dibujo del puente.

3.4 MensajeEngine

Clasificador de estilo con precedencia explícita:

```
estilos.any { AGRESIVO }      -> AGRESIVO
estilos.count { PASIVO } >= 2 -> PASIVO
estilos.all { ASERTIVO }      -> ASERTIVO
estilos.count { ASERTIVO } >= 3 -> ASERTIVO
else                          -> PASIVO
```

Puntuación = estructura (55 puntos repartidos entre las cuatro ranuras) + tono (+12 por ficha asertiva, −14 por agresiva, −5 por pasiva). `construir()` devuelve la frase montada, el estilo y una lista de observaciones que señalan qué parte falla.

3.5 ConflictoEngine

Máquina de estados pura:

```
aplicar(estado, opcion) = estado.copy(
    calma    = (estado.calma + opcion.dCalma).coerceIn(0, 100),
    confianza = ...,
    acuerdo  = ...,
    turno    = estado.turno + 1,
    nodoId   = opcion.siguienteNodo ?: estado.nodoId
)
```

Umbral de desenlace: ruptura si calma < 30 || confianza < 30; acuerdo si acuerdo >= 65 && calma >= 50; tregua en el resto. Puntuación: acuerdo·0.45 + calma·0.30 + confianza·0.25.

3.6 TermometroEngine

Dos mitades independientes: la precisión de la medida (con margen configurable por reto, y caída lineal de 5 unidades hasta cero) y la adecuación de la estrategia, evaluada **contra la intensidad real** de la escena, no contra la que el jugador estimó.

4. Progresión, desbloques e insignias

domain/usecase/Progreso.kt:

- `xpParaSubir(nivel) = 100 + (nivel - 1) · 50`, hasta nivel 12.
- `xpGanada(mision, estrellas, yaCompletada)`: 100 % / 75 % / 50 % / 5 XP según estrellas, y un

factor 0.35 si ya estaba completada.

- `DesbloqueoEvaluador.estadoMision` devuelve ``BLOQUEADA / DISPONIBLE / INICIADA / COMPLETADA /`

DOMINADA` combinando el desbloqueo de zona (por XP) con la cadena de misiones dentro de la zona.

- `siguienteMision` recorre las zonas en orden y devuelve la primera disponible; si no queda

ninguna, propone la de menos estrellas.

`InsigniaEvaluador` es una lista de 12 `ReglaInsignia`, cada una con un objetivo numérico y un extractor sobre `EstadisticasJugador`. `nuevas(antes, ahora)` compara dos fotografías del jugador, que es como el repositorio detecta qué insignias acaban de ganarse.

5. Capa de datos

5.1 Room

Siete entidades, versión 1, esquema exportado a `app/schemas/`. Detalle completo en `docs/BASE_DE_DATOS.md`.

Los DAO se declaran como **interfaces**, lo que permite que las pruebas JVM las implementen con dobles en memoria sin tocar Android ni Robolectric.

5.2 JuegoRepository

Expone un único `Flow<EstadoJuego>` construido con combine de cinco fuentes (perfil, progreso, cartas, insignias y un agregado de ánimos, visitas y hitos por mecánica). La interfaz nunca consulta un DAO directamente.

`registrarResultado(misionId, resultado, hito)` es el corazón del bucle y hace, en orden:

1. Fotografía las estadísticas **antes**.
2. Actualiza `progreso_mision` guardando el máximo de estrellas y de puntaje.
3. Inserta el intento en el historial, con su marca de hito.
4. Suma XP al perfil y registra la visita del día.
5. Desbloquea la carta si la misión se completa por primera vez.

6. Fotografía las estadísticas **después** y otorga las insignias nuevas.
7. Comprueba si la zona ha quedado completa.
8. Devuelve un Recompensa con todo lo ganado.

El **hito** es el logro propio de cada mecánica: 3 estrellas en rostro o escucha, tres tabloncillos correctos, mensaje asertivo completo con ≥ 88 puntos, o acuerdo con calma ≥ 50 .

5.3 AjustesRepository

DataStore Preferences con cinco interruptores: sonido, vibración, animaciones, texto grande y alto contraste. Se inyectan en la interfaz mediante LocalAjustes.

5.4 Tiempo

util/Relej.kt abstrae el reloj tras una interfaz con dos implementaciones: RelejSistema y RelejFijo (usada en pruebas para simular días). Se evita java.time porque minSdk es 24; los días se calculan como `floorDiv(millis + offsetZona, 86_400_000)`.

6. Capa de interfaz

6.1 Arte con Canvas

No hay ninguna imagen de mapa de bits salvo el icono del launcher. Todo lo demás se dibuja:

Archivo	Contenido
art/Nima.kt	La guía, con seis estados y animación de flotación y parpadeo
art/Rostro.kt	Rostro paramétrico: los cuatro ejes se traducen en geometría
art/Avatares.kt	8 avatares con accesorio distinto
art/Figuras.kt	12 símbolos vectoriales para cartas, insignias e iconos
art/Escenarios.kt	Fondo de isla animado y 6 escenas de zona diferenciadas

6.2 Arrastrar y soltar

components/Arrastre.kt implementa un sistema propio:

- CapaArrastre envuelve toda la aplicación y dibuja la pieza flotante en una capa superior.
- Modifier.zonaSoltar(clave, estado) registra los límites del destino en coordenadas de ventana

mediante `onGloballyPositioned + positionInWindow`.

- Modifier.piezaArrastrable(dato, vistaPrevia, alSoltar) usa

`detectDragGesturesAfterLongPress` para no competir con el scroll vertical.

Toda actividad que use arrastre ofrece además la alternativa por toque.

6.3 ViewModels

- JuegoViewModel — estado global, ajustes, diario, racha, siguiente misión y misiones de repaso,

todo como `StateFlow` con `SharingStarted.WhileSubscribed(5s)`.

- ActividadViewModel — estado de la misión en curso, creado con clave `actividad_<misionId>`.

Guarda la configuración del rostro, las colocaciones, el estado del conflicto y la bitácora; evaluar () delega en el motor que corresponda y calcula el hito.

6.4 Navegación

14 rutas en `ui/navigation/Navegacion.kt`, con transiciones de deslizamiento y fundido. Las rutas con parámetro (`zona/{zonaId}`, `misión/{misiónId}`) validan el argumento y caen a un valor por defecto si no es válido.

6.5 Tema

`SocialKidsTheme` construye el esquema de color a partir de la paleta propia y aplica dos ajustes de accesibilidad: escala tipográfica $\times 1.18$ con texto grande y esquemas planos con alto contraste. Modo claro y oscuro completos.

7. Retroalimentación

`util/Retroalimentacion.kt` combina háptica y sonido, ambos opcionales:

- Háptica mediante `LocalHapticFeedback`, que **no requiere el permiso** `VIBRATE`.
- Sonido mediante `ToneGenerator` del sistema: tonos cortos, sin archivos de audio y sin peso extra

en el APK. Es una simplificación consciente frente a un diseño sonoro propio.

8. Pruebas

8.1 Unitarias (JVM) — 93 pruebas

Clase	Pruebas	Cubre
<code>RostroEngineTest</code>	7	Puntuación por eje, extras, límites, pista
<code>EscuchaEngineTest</code>	6	F1, datos inventados, calidad de respuesta
<code>PuenteEngineTest</code>	5	Aciertos, distractoras, integridad del contenido
<code>MensajeEngineTest</code>	7	Clasificador de estilo, estructura, frase montada
<code>ConflictoEngineTest</code>	7	Recorridos completos, límites, grafo de nodos
<code>TermometroEngineTest</code>	6	Margen, proporcionalidad, rangos imposibles
<code>EstrellasTest</code>	1	Cortes de estrellas
<code>ProgresoCalculadoraTest</code>	8	Curva de niveles, XP repetida, valores negativos
<code>DesbloqueoEvaluadorTest</code>	8	Cadena de misiones, porcentajes, repaso
<code>InsigniaEvaluadorTest</code>	7	Reglas, insignias nuevas, siguiente objetivo
<code>EstadisticasTest</code>	7	Medias, series de 7 días, rachas
<code>ContenidoTest</code>	10	Integridad del catálogo y proporción de mecánicas
<code>JuegoRepositoryTest</code>	14	Bucle completo con DAO falsos

Casos límite cubiertos: listas vacías, selección vacía, XP negativa, valores fuera de rango, texto demasiado largo, alias en blanco, misión inexistente, doble desbloqueo de carta, repetición de misión, racha con hueco y reinicio total.

```
./gradlew testDebugUnitTest
```

8.2 Instrumentadas (Room) — 7 pruebas

app/src/androidTest/.../BaseDeDatosTest.kt valida la persistencia real: upsert sin duplicados, conteo de hitos por misión distinta, cartas y visitas únicas, orden del diario y borrado total. Requieren dispositivo o emulador:

```
./gradlew connectedDebugAndroidTest
```

9. Compilación

```
./gradlew clean
./gradlew testDebugUnitTest
./gradlew lintDebug
./gradlew assembleDebug
./gradlew assembleRelease # sin firmar
```

Requiere local.properties con sdk.dir apuntando al SDK de Android. Ese archivo está en .gitignore y no debe subirse al repositorio.

Integración continua

.github/workflows/android.yml ejecuta la misma secuencia en ubuntu-latest con JDK 17 y SDK 35, publica APK, fuentes, informes y PDF como artefactos, y crea una release de GitHub al empujar una etiqueta v*.

10. Extender la aplicación

Añadir una misión: crear la entrada en MundoSeed.misiones, añadir la carta correspondiente en CartasSeed y el reto en el archivo de contenido de su mecánica. ContenidoTest fallará si algo queda descolgado (carta inexistente, orden mal numerado, contenido ausente).

Añadir una mecánica: nuevo valor en Mecanica, motor en domain/engine con su evaluar(...): ResultadoActividad, estado en ActividadViewModel, rama en PantallaMision y pantalla en ui/screens.

Añadir una insignia: nueva ReglaInsignia en InsigniaEvaluador. Si necesita un dato que aún no se guarda, añadirlo a EstadisticasJugador y calcularlo en el repositorio.

Cambiar el esquema de la base de datos: subir la versión en SocialKidsDatabase y escribir la migración. En desarrollo está activo fallbackToDestructiveMigration(), que borra los datos; para producción debe sustituirse por migraciones reales.